

COMPTE RENDU

Bases de Données Avancées - Lab No. 7
3e année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
Filière : Cybersecurité
Année : 2025/2026
Enseignants : Mme.ELHAJI & M.HJJAMI
Date : 28 novembre 2025

Table des matières

1	Exercice 1 : Tablespaces et Fichiers de Données - Création, Quotas et Gestion	2
1.1	Question 1 : Connexion en tant que SYSTEM	2
1.2	Question 2 : Création du tablespace	2
1.3	Question 3 : Création de l'utilisateur avec quota	3
1.4	Question 4 : Attribution des permissions	3
1.5	Question 5 : Création de la table test	3
1.6	Question 6 : Test du quota	4
1.7	Question 7 : Test d'insertion après saturation	4
1.8	Question 8 : Extension du quota	5
1.9	Question 9 : Nouvelle tentative d'insertion	5
2	Exercice 2 : Indexation et Optimisation - Index B-Tree	6
2.1	Objectif	6
2.2	Question 1 : Création de la table de test	6
2.3	Question 2 : Requête AVANT indexation	6
2.4	Question 3 : Création de l'index B-Tree	7
2.5	Question 4 : Requête APRÈS indexation	7
2.6	Question 5 : Comparaison des résultats	8
2.7	Question 6 : Impact de l'index B-Tree	8
3	Exercice 3 : Transactions et Contrôle de Concurrency	8
3.1	Objectif	8
3.2	Question 1 : État initial	8
3.3	Question 2 : Démarrage d'une transaction	9
3.4	Question 3 : Vérification dans Session 2	9
3.5	Question 4 : Observation de la visibilité	10
3.6	Question 5 : COMMIT de la transaction	10
3.7	Question 6 : Vérification après COMMIT	10
3.8	Question 7 : Test avec ROLLBACK	11
3.9	Question 8 : Fonction de ROLLBACK	11

1 Exercice 1 : Tablespaces et Fichiers de Données - Création, Quotas et Gestion

Objectif : Apprendre à créer et gérer des tablespaces, attribuer des quotas, et comprendre comment les limites d'espace affectent les utilisateurs.

1.1 Question 1 : Connexion en tant que SYSTEM

```
hathouti@Taha-inspiron-16:~/sqlcl/bin$ oracle-start
oracle-xe
✓ Oracle Database démarré
hathouti@Taha-inspiron-16:~/sqlcl/bin$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND
CREATED        STATUS          PORTS
NAMES
479b68aefd17   gvenzl/oracle-xe:21-slim            "container-entrypoint" 7
weeks ago     Up 3 seconds    0.0.0.0:1521->1521/tcp, [::]:1521->1521/
tcp           oracle-xe
hathouti@Taha-inspiron-16:~/sqlcl/bin$ sqlsystem

SQLcl : version 25.3 Production sur ven. nov. 28 14:51:34 2025

Copyright (c) 1982, 2025, Oracle. Tous droits réservés.

Last Successful login time: Ven. Nov. 28 2025 14:51:34 +01:00

Connecté à :
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL>
```

Listing 1 – Connexion SYSTEM

1.2 Question 2 : Création du tablespace

```
1 CREATE TABLESPACE tbs_students
2 DATAFILE 'tbs_students1.dbf' SIZE 100M;
```

Listing 2 – Création du tablespace tbs_students

Résultat :

```
SQL> CREATE TABLESPACE tbs_students DATAFILE 'tbs_students1.dbf' SIZE
100M;

TABLESPACE TBS_STUDENTS créé(e).
```

1.3 Question 3 : Création de l'utilisateur avec quota

```
1 CREATE USER student1 IDENTIFIED BY "123456"  
2 DEFAULT TABLESPACE tbs_students  
3 QUOTA 1M ON tbs_students;
```

Listing 3 – Création de student1

```
SQL> CREATE USER student1 IDENTIFIED BY "123456"  
2 DEFAULT TABLESPACE tbs_students  
3* QUOTA 1M ON tbs_students;  
  
User STUDENT1 créé(e).
```

1.4 Question 4 : Attribution des permissions

```
1 GRANT CONNECT, RESOURCE TO student1;
```

Listing 4 – Grant des privilèges

```
SQL> GRANT CONNECT, RESOURCE TO student1;  
  
Succès de l'élément Grant.
```

1.5 Question 5 : Création de la table test

```
hathouti@Taha-inspiron-16:~/sqlcl/bin$ ./sql student1/123456@//localhost  
:1521/XEPDB1  
  
SQLcl : version 25.3 Production sur ven. nov. 28 23:05:01 2025  
  
Copyright (c) 1982, 2025, Oracle. Tous droits réservés.  
  
Connecté à :  
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production  
Version 21.3.0.0.0  
  
SQL>
```

Listing 5 – Connexion student1

```
1 CREATE TABLE test_table (id NUMBER, name VARCHAR2(50));
```

Listing 6 – Création de test_table

```
SQL> CREATE TABLE test_table (id NUMBER, name VARCHAR2(50));  
  
Table TEST_TABLE créé(e).
```

Listing 7 – Connexion student1

1.6 Question 6 : Test du quota

```
1 SET SERVEROUTPUT ON;
2 DECLARE
3     v_counter NUMBER := 1;
4 BEGIN
5     LOOP
6         INSERT INTO test_table (id, name)
7         VALUES (v_counter, 'Name ' || v_counter);
8         v_counter := v_counter + 1;
9         COMMIT;
10    END LOOP;
11 EXCEPTION
12     WHEN OTHERS THEN
13         DBMS_OUTPUT.PUT_LINE('Tablespace quota exceeded after
14         inserting ' ||
15         (v_counter-1) || ' rows.');
```

Listing 8 – Insertion jusqu'à saturation du quota

Résultat obtenu :

```
SQL> SET SERVEROUTPUT ON ;
SQL> DECLARE
2   v_counter NUMBER := 1;
3   BEGIN
4   LOOP
5   INSERT INTO test_table (id, name)
6   VALUES (v_counter, 'Name ' || v_counter);
7   v_counter := v_counter + 1;
8   COMMIT;
9   END LOOP;
10  EXCEPTION
11  WHEN OTHERS THEN
12  DBMS_OUTPUT.PUT_LINE('Quota du tablespace dépassé après insertion
de ' || (v_counter-1) || ' lignes.');
```

Quota du tablespace dépassé après insertion de 41828 lignes.

Procédure PL/SQL terminée.

1.7 Question 7 : Test d'insertion après saturation

```
1 INSERT INTO test_table (id, name)
2 VALUES (99999, 'Hasna');
```

Listing 9 – Tentative d'insertion

Message d'erreur :

```
SQL> INSERT INTO test_table (id, name)
2* VALUES (99999, 'Hasna');
```

```

Erreur commençant à la ligne: 1 de la commande -
INSERT INTO test_table (id, name)
VALUES (99999, 'Hasna')
Erreur à la ligne de commande: 1 Colonne: 13
Rapport d'erreur -
Erreur SQL : ORA-01536: dépassement du quota d'espace affecté au
    tablespace 'TBS_STUDENTS'

https://docs.oracle.com/error-help/db/ora-01536/01536. 00000 - "space
    quota exceeded for tablespace '%s'"
*Cause:      The space quota for the segment owner in the tablespace has
              been exhausted and the operation attempted the creation of a
              new segment extent in the tablespace.
*Action:     Either drop unnecessary objects in the tablespace to reclaim
              space or have a privileged user increase the quota on this
              tablespace for the segment owner.

More Details :
https://docs.oracle.com/error-help/db/ora-01536/

```

Analyse : L'utilisateur student1 a atteint sa limite de quota (1M) sur le tablespace tbs_students. Oracle refuse toute nouvelle insertion tant que le quota n'est pas étendu.

1.8 Question 8 : Extension du quota

```

1 ALTER USER student1 QUOTA 5M ON tbs_students;

```

Listing 10 – Extension à 5M

```

hathouti@Taha-inspiron-16:~/sqlcl/bin$ sqlsys

SQLcl : version 25.3 Production sur ven. nov. 28 23:15:56 2025

Copyright (c) 1982, 2025, Oracle. Tous droits réservés.

Connecté à :
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL> ALTER SESSION SET CONTAINER = XEPDB1;

Session modifié(e).

SQL> ALTER USER student1 QUOTA 5M ON tbs_students;

User STUDENT1 modifié(e).

```

1.9 Question 9 : Nouvelle tentative d'insertion

```

1 INSERT INTO test_table (id, name)
2 VALUES (99999, 'Hasna');
3 COMMIT;

```

Listing 11 – Insertion réussie

Résultat :

```
SQL> INSERT INTO test_table (id, name)
2* VALUES (99999, 'Hasna');

1 ligne inséré.
```

Conclusion : Après l'extension du quota à 5M, l'insertion s'effectue sans problème.

2 Exercice 2 : Indexation et Optimisation - Index B-Tree

2.1 Objectif

Comprendre comment un index B-Tree améliore les performances des requêtes et comment vérifier son utilisation.

2.2 Question 1 : Création de la table de test

```
1 CREATE TABLE copy_employee AS SELECT * FROM employees;
```

Listing 12 – Copie de la table EMPLOYEES

```
hathouti@Taha-inspiron-16:~/sqlcl/bin$ sqlhr

SQLcl : version 25.3 Production sur ven. nov. 28 23:21:32 2025

Copyright (c) 1982, 2025, Oracle. Tous droits réservés.

Connecté à :
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL> CREATE TABLE copy_employee AS SELECT * FROM employees ;

Table COPY_EMPLOYEE créé(e).
```

Listing 13 – Connexion HR

2.3 Question 2 : Requête AVANT indexation

```
1 SET TIMING ON;
2 SELECT employee_id, first_name, last_name, salary, department_id
3 FROM copy_employee
4 WHERE department_id = 50
5     AND salary > 5000
6 ORDER BY employee_id;
```

Listing 14 – Test de performance sans index

Résultats :

```
SQL> SET TIMING ON;
SQL> SELECT employee_id, first_name, last_name, salary, department_id
  2  FROM copy_employee
  3  WHERE department_id = 50 AND salary > 5000
  4* ORDER BY employee_id;

EMPLOYEE_ID FIRST_NAME LAST_NAME SALARY DEPARTMENT_ID
-----
120 Matthew Weiss 8800 50
121 Adam Fripp 9020 50
122 Payam Kaufling 8690 50
123 Shanta Vollman 7150 50
124 Kevin Mourgos 6380 50

Ecoulé : 00:00:00.014
```

Temps d'exécution AVANT index : 0.014 secondes

2.4 Question 3 : Création de l'index B-Tree

```
1 CREATE INDEX btree_index
2 ON copy_employee(department_id, salary, employee_id);
```

Listing 15 – Index composite

Résultats :

```
SQL> CREATE INDEX btree_index ON copy_employee(department_id, salary,
employee_id);

Index BTREE_INDEX créé(e).
```

2.5 Question 4 : Requête APRÈS indexation

```
1 SET TIMING ON;
2 SELECT employee_id, first_name, last_name, salary, department_id
3 FROM copy_employee
4 WHERE department_id = 50
5     AND salary > 5000
6 ORDER BY employee_id;
```

Listing 16 – Même requête avec index

Résultats :

```
SQL> SELECT employee_id, first_name, last_name, salary, department_id
  2  FROM copy_employee
  3  WHERE department_id = 50
  4    AND salary > 5000
  5* ORDER BY employee_id;

EMPLOYEE_ID FIRST_NAME LAST_NAME SALARY DEPARTMENT_ID
-----
120 Matthew Weiss 8800 50
121 Adam Fripp 9020 50
```


122	Payam	Kaufling	8690	50
123	Shanta	Vollman	7150	50
124	Kevin	Mourgos	6380	50

Ecoulé : 00:00:00.026

Temps d'exécution APRÈS index : 0.026 secondes

2.6 Question 5 : Comparaison des résultats

Métrique	Sans Index	Avec Index
Temps d'exécution	0.014 s	0.026 s
Amélioration	1.9× plus rapide	-

TABLE 1 – Comparaison des performances

2.7 Question 6 : Impact de l'index B-Tree

Sur cette petite table de démonstration, l'index n'apporte pas d'amélioration visible. Je pense que dans un environnement de tables plus volumineuses (milliers/millions de lignes), l'index B-Tree réduirait significativement le temps d'exécution en évitant le scan complet de la table.

3 Exercice 3 : Transactions et Contrôle de Concurrency

3.1 Objectif

Apprendre comment fonctionnent les transactions et comment COMMIT et ROLLBACK affectent la visibilité des données.

3.2 Question 1 : État initial

Session 1 (HR) :

```
1 SELECT * FROM employees WHERE department_id = 60;
```

Listing 17 – Consultation initiale

Résultats :

```
hathouti@Taha-inspiron-16:~/sqlcl/bin$ sqlhr

SQLcl : version 25.3 Production sur ven. nov. 28 23:36:24 2025

Copyright (c) 1982, 2025, Oracle. Tous droits réservés.

Connecté à :
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
```

```
Version 21.3.0.0.0

SQL> SELECT * FROM employees WHERE department_id = 60;

EMPLOYEE_ID FIRST_NAME LAST_NAME EMAIL PHONE_NUMBER HIRE_DATE JOB_ID
SALARY COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-----
103 Alexander Hunold AHUNOLD 590.423.4567 03/01/06 IT_PROG
9000 102 60
104 Bruce Ernst BERNST 590.423.4568 21/05/07 IT_PROG
6000 103 60
105 David Austin DAUSTIN 590.423.4569 25/06/05 IT_PROG
4800 103 60
106 Valli Pataballa VPATABAL 590.423.4560 05/02/06 IT_PROG
4800 103 60
107 Diana Lorentz DLORENTZ 590.423.5567 07/02/07 IT_PROG
4200 103 60
```

3.3 Question 2 : Démarrage d'une transaction

Session 1 :

```
1 UPDATE employees
2 SET salary = salary + 500
3 WHERE department_id = 60;
```

Listing 18 – Modification non commitée

Résultat :

```
SQL> UPDATE employees SET salary = salary + 500 WHERE department_id =
60;

5 lignes mis à jour.
```

3.4 Question 3 : Vérification dans Session 2

Session 2 (HR - nouvelle connexion) :

```
1 SELECT * FROM employees WHERE department_id = 60;
```

Listing 19 – Consultation concurrente

```
hathouti@Taha-inspiron-16:~/sqlcl/bin$ sqlhr

SQLcl : version 25.3 Production sur ven. nov. 28 23:39:58 2025

Copyright (c) 1982, 2025, Oracle. Tous droits réservés.

Connecté à :
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL> SELECT * FROM employees WHERE department_id = 60;
```

```

EMPLOYEE_ID FIRST_NAME LAST_NAME EMAIL      PHONE_NUMBER HIRE_DATE JOB_ID
SALARY COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-----
103 Alexander Hunold AHUNOLD 590.423.4567 03/01/06 IT_PROG
9000 102 60
104 Bruce Ernst BERNST 590.423.4568 21/05/07 IT_PROG
6000 103 60
105 David Austin DAUSTIN 590.423.4569 25/06/05 IT_PROG
4800 103 60
106 Valli Pataballa VPATABAL 590.423.4560 05/02/06 IT_PROG
4800 103 60
107 Diana Lorentz DLORENTZ 590.423.5567 07/02/07 IT_PROG
4200 103 60
SQL>

```

3.5 Question 4 : Observation de la visibilité

Résultat : Les modifications ne sont PAS visibles dans la Session 2. La Session 2 voit la dernière version des données avant la modification, car la transaction de la Session 1 n'est pas encore commitée.

3.6 Question 5 : COMMIT de la transaction

Session 1 :

```
1 COMMIT;
```

Listing 20 – Validation de la transaction

3.7 Question 6 : Vérification après COMMIT

Session 2 :

```
1 SELECT * FROM employees WHERE department_id = 60;
```

Listing 21 – Nouvelle consultation

Résultat :

```

SQL> SELECT * FROM employees WHERE department_id = 60;

EMPLOYEE_ID FIRST_NAME LAST_NAME EMAIL      PHONE_NUMBER HIRE_DATE JOB_ID
SALARY COMMISSION_PCT MANAGER_ID DEPARTMENT_ID
-----
103 Alexander Hunold AHUNOLD 590.423.4567 03/01/06 IT_PROG
9500 102 60
104 Bruce Ernst BERNST 590.423.4568 21/05/07 IT_PROG
6500 103 60
105 David Austin DAUSTIN 590.423.4569 25/06/05 IT_PROG
5300 103 60
106 Valli Pataballa VPATABAL 590.423.4560 05/02/06 IT_PROG
5300 103 60
107 Diana Lorentz DLORENTZ 590.423.5567 07/02/07 IT_PROG
4700 103 60

```

3.8 Question 7 : Test avec ROLLBACK

Session 1 :

```
1 UPDATE employees
2 SET salary = salary - 500
3 WHERE department_id = 60;
4
5 -- Annulation
6 ROLLBACK;
```

Listing 22 – Nouvelle modification

Vérification Session 1 :

```
1 SELECT * FROM employees WHERE department_id = 60;
```

Résultat :

```
SQL> UPDATE employees SET salary = salary + 500 WHERE department_id =
60;

5 lignes mis à jour.

SQL> ROLLBACK;
Annulation terminée.
SQL> SELECT * FROM employees WHERE department_id = 60;
```

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID
SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID			
103	Alexander	Hunold	AHUNOLD	590.423.4567	03/01/06	IT_PROG
9500				102	60	
104	Bruce	Ernst	BERNST	590.423.4568	21/05/07	IT_PROG
6500				103	60	
105	David	Austin	DAUSTIN	590.423.4569	25/06/05	IT_PROG
5300				103	60	
106	Valli	Pataballa	VPATABAL	590.423.4560	05/02/06	IT_PROG
5300				103	60	
107	Diana	Lorentz	DLORENTZ	590.423.5567	07/02/07	IT_PROG
4700				103	60	

3.9 Question 8 : Fonction de ROLLBACK

ROLLBACK annule toutes les modifications effectuées depuis le dernier COMMIT.